

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

Doctor Byte (FASE 1)

PONDERACIÓN: 20 pts

 **Tiempo estimado: 30 hrs**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	4
2.1. Objetivo SMART.....	4
3. Resumen Ejecutivo.....	5
4. Enunciado del Proyecto.....	5
4.1 Descripción del problema o necesidad a resolver.....	5
4.2 Alcance del proyecto.....	6
4.3 Recursos y herramientas a utilizar.....	6
4.4 Entregables.....	8
5. Material de apoyo.....	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad	Cada integrante debe cumplir con las tareas asignadas y entregar componentes funcionales dentro de los plazos establecidos.
Trabajo	Los estudiantes trabajarán en el desarrollo del sistema, integrando Prolog, frontend, backend y bot de Telegram.
Innovación	Se fomenta la búsqueda de soluciones tecnológicas para automatizar el diagnóstico de fallas en computadoras mediante inteligencia artificial.
Compromiso	Los participantes deben asegurar la calidad y correcto funcionamiento del proyecto antes de su entrega.

1.2. Competencia(s)

Diseñar e implementar un sistema experto utilizando técnicas de Inteligencia Artificial basadas en reglas y lógica declarativa, integrando herramientas de desarrollo de software para resolver problemas reales mediante la automatización del diagnóstico de fallas en computadoras.

1.3. Habilidad(es) blandas a formar

El proyecto permitirá desarrollar las siguientes habilidades:

- Comunicación efectiva.
- Pensamiento analítico.
- Resolución de problemas.
- Gestión del tiempo.
- Aprendizaje autónomo.
- Adaptabilidad tecnológica.
- Toma de decisiones basada en evidencia.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Desarrollar un sistema experto denominado Doctor Byte capaz de diagnosticar fallas comunes en computadoras mediante reglas implementadas en Prolog e integradas con una interfaz web y un bot de Telegram para la comunicación de resultados.	Implementar al menos: 15 síntomas. 10 fallas diagnosticables. 10 recomendaciones. 10 reglas de inferencia. 1 interfaz web funcional. 1 integración con Telegram para notificación de diagnósticos.	Utilizando Prolog para la construcción de la base de conocimiento, un backend para procesar consultas, una interfaz web para la interacción del usuario y un bot de Telegram para el envío automático de resultados.	Para aplicar conceptos fundamentales de Inteligencia Artificial, lógica declarativa e integración de sistemas en la solución de un problema real relacionado con el diagnóstico de fallas informáticas.	El proyecto deberá desarrollarse y entregarse en un período máximo de 15 días calendario , conforme al cronograma establecido para la Fase 1 del curso.

3. Resumen Ejecutivo

El proyecto Doctor Byte consiste en el desarrollo de un sistema experto orientado al diagnóstico de fallas comunes en computadoras mediante el uso de técnicas fundamentales de Inteligencia Artificial. La solución permitirá que un usuario seleccione una serie de síntomas relacionados con el comportamiento de un equipo de cómputo y reciba un diagnóstico junto con recomendaciones básicas para la posible solución del problema.

Para la implementación del sistema se utilizará Prolog como motor de inferencia, aprovechando sus capacidades para representar conocimiento mediante hechos, reglas y consultas. Adicionalmente, se desarrollará una interfaz web que facilite la interacción con el usuario y un bot de Telegram que permite notificar los resultados obtenidos de manera automática.

El proyecto busca que los estudiantes apliquen conceptos de lógica declarativa, sistemas expertos e integración de tecnologías, fortaleciendo tanto sus conocimientos técnicos como habilidades de trabajo colaborativo, análisis y resolución de problemas. Como resultado, se obtendrá una herramienta funcional capaz de automatizar el proceso de diagnóstico preliminar de fallas informáticas, simulando el comportamiento de un sistema experto basado en reglas.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

El diagnóstico de fallas en computadoras suele requerir conocimientos técnicos especializados para identificar correctamente las posibles causas de un problema. En muchos casos, usuarios con poca experiencia informática desconocen cómo interpretar síntomas como pantallas negras, reinicios inesperados, mensajes de error, lentitud del sistema o problemas de arranque, lo que dificulta la toma de decisiones para resolver la situación.

Ante esta necesidad, se propone el desarrollo de **Doctor Byte**, un sistema experto capaz de analizar síntomas proporcionados por el usuario y generar un diagnóstico preliminar acompañado de recomendaciones básicas. El sistema utilizará reglas de inferencia implementadas en Prolog para simular el razonamiento de un experto en soporte técnico, facilitando la identificación de posibles fallas de hardware o software de forma automatizada.

4.2 Alcance del proyecto

Alcance obligatorio:

El proyecto deberá incluir como mínimo los siguientes componentes:

- Desarrollo de una base de conocimiento en Prolog.
- Implementación de al menos 15 síntomas relacionados con fallas comunes en computadoras.
- Implementación de al menos 10 fallas diagnosticables.
- Implementación de al menos 10 recomendaciones asociadas a las fallas identificadas.
- Uso de hechos, reglas, variables, listas y al menos un corte (!) dentro de la solución en Prolog.
- Desarrollo de una interfaz web que permita seleccionar síntomas y solicitar un diagnóstico.
- Desarrollo de un backend con python que procese las solicitudes y se comunique con el motor de inferencia en Prolog.
- Generación de diagnósticos basados en las reglas definidas.
- Integración con un bot de Telegram para enviar notificaciones de los diagnósticos realizados.
- Documentación técnica básica del proyecto.
- Presentación funcional del sistema durante la evaluación.
- Historial de diagnósticos realizados.

Alcance opcional:

- Implementación de contenedores Docker.
- Despliegue en servicios de nube como Azure, AWS o Google Cloud.
- Integración de pruebas unitarias automatizadas.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Software	SWI-Prolog para la implementación del sistema experto basado en hechos, reglas y consultas.
Obligatorio	Software	Lenguaje de programación Python o Node.js para el desarrollo del backend y la integración con Prolog.
Obligatorio	Software	Framework web (Flask, FastAPI)

		o Express) para la creación de la API de comunicación entre el frontend y el motor de inferencia.
Obligatorio	Software	Visual Studio Code o editor de código equivalente para el desarrollo del proyecto.
Obligatorio	Software	Git para el control de versiones del código fuente.
Obligatorio	Plataforma	GitHub para el almacenamiento y gestión del repositorio del proyecto.
Obligatorio	Plataforma	Telegram Bot API para el envío de notificaciones de diagnósticos realizados por el sistema.
Obligatorio	Software	Navegador web moderno (Chrome, Edge o Firefox) para la ejecución y pruebas de la interfaz.
Obligatorio	Hardware	Computadora con acceso a Internet para el desarrollo, pruebas y presentación del proyecto.
Obligatorio	Software	Uso de frameworks para interfaz
Opcional	Software	Docker para la contenerización de la aplicación.
Opcional	Plataforma	Azure, AWS o Google Cloud para el despliegue de la solución en la nube.
Opcional	Software	Postman para pruebas y validación de los servicios REST.
Opcional	Software	PyTest para la creación y ejecución de pruebas unitarias sobre el backend y la integración del sistema

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Repositorio GitHub	Repositorio con el código fuente completo del proyecto, incluyendo historial de commits y documentación básica de instalación y ejecución.
Base de conocimiento Prolog	Archivo(s) .pl que contengan hechos, reglas, listas, consultas y cortes utilizados para realizar los diagnósticos de fallas en computadoras.
Frontend web	Interfaz gráfica que permita al usuario seleccionar síntomas y visualizar el diagnóstico generado por el sistema.
Backend	API o servicio encargado de recibir solicitudes, comunicarse con Prolog y retornar los resultados al usuario.
Integración Telegram	Implementación de un bot o servicio de notificación mediante Telegram que informe los diagnósticos realizados.
Documento técnico	Documento que describa la arquitectura, tecnologías utilizadas, estructura del proyecto y funcionamiento general de la solución.
Manual de usuario	Guía breve que explique cómo ejecutar y utilizar el sistema.
Evidencias de funcionamiento	Capturas de pantalla o registros que demuestren el correcto funcionamiento de los principales casos de uso.
Video demostrativo	Video de entre 5 y 10 minutos mostrando la ejecución del sistema, las funcionalidades implementadas y una explicación general de la solución.
Presentación final	Presentación utilizada para exponer el proyecto, incluyendo problema, solución propuesta, arquitectura y resultados obtenidos.
Diagrama de arquitectura	Representación gráfica de los componentes del sistema y su interacción (Frontend, Backend, Prolog y Telegram).
Casos de prueba	Documento con los escenarios de prueba ejecutados y los resultados obtenidos durante la validación del proyecto.

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	Link	Descripción
Documentación oficial	https://www.swi-prolog.org	Sitio oficial de SWI-Prolog. Incluye instalación, sintaxis, ejemplos y manual de referencia.
Manual	https://www.swi-prolog.org/pldoc/man?section=quickstart	Guía rápida para iniciar el desarrollo de programas en Prolog.
Documentación oficial	https://developer.mozilla.org	Documentación de HTML, CSS y JavaScript utilizada para el desarrollo del frontend.
Documentación oficial	https://flask.palletsprojects.com	Documentación oficial de Flask para el desarrollo de APIs REST en Python.
Ejemplo	https://realpython.com/flask-con-nexion-rest-api	Ejemplo de construcción de servicios REST utilizando Flask.
Documentación oficial	https://core.telegram.org/bots	Documentación oficial de la API de Bots de Telegram.
Ejemplo	https://python-telegram-bot.org	Librería para la integración de bots de Telegram utilizando Python.
Documentación oficial	https://git-scm.com/doc	Documentación oficial de Git para control de versiones.
Plataforma	https://github.com	Plataforma para alojamiento del código fuente y colaboración en el proyecto.
Tutorial	https://www.youtube.com/results?search_query=swi+prolog+tutorial	Recursos audiovisuales para el aprendizaje de Prolog mediante ejemplos prácticos.
Tutorial	https://www.youtube.com/results?search_query=flask+rest+api+tutorial	Videos de apoyo para la creación de APIs REST utilizando Flask.

**El link es opcional para una mejor guía del material de apoyo

Fecha de entrega: 12/06/2026

Repositorio: El mismo de la práctica, en una carpeta diferente